



Security Audit

Silverswap (DCA)

Table of Contents

Executive Summary	4
Project Context	4
Audit scope	7
Security Rating	8
Intended Smart Contract Behaviours	9
Code Quality	9
Audit Resources	10
Dependencies	10
Severity Definitions	11
Audit Findings	12
Centralisation	18
Conclusion	19
Our Methodology	20
Disclaimers	22
About Hashlock	23

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Silverswap team partnered with Hashlock to conduct a security audit of their protocol smart contracts' changes. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts were secure.

Project Context

Silverswap is a DeFi dApp built on Fantom Opera Chain. Dollar-cost averaging (DCA) is a popular investment strategy in the crypto world that allows you to invest a fixed amount of money at regular intervals, regardless of market conditions.

Simply put, this strategy helps reduce the impact of market volatility and removes the need to time the market.

Project Name: Silverswap

Compiler Version: $\geq 0.8.20 \leq 0.8.25$

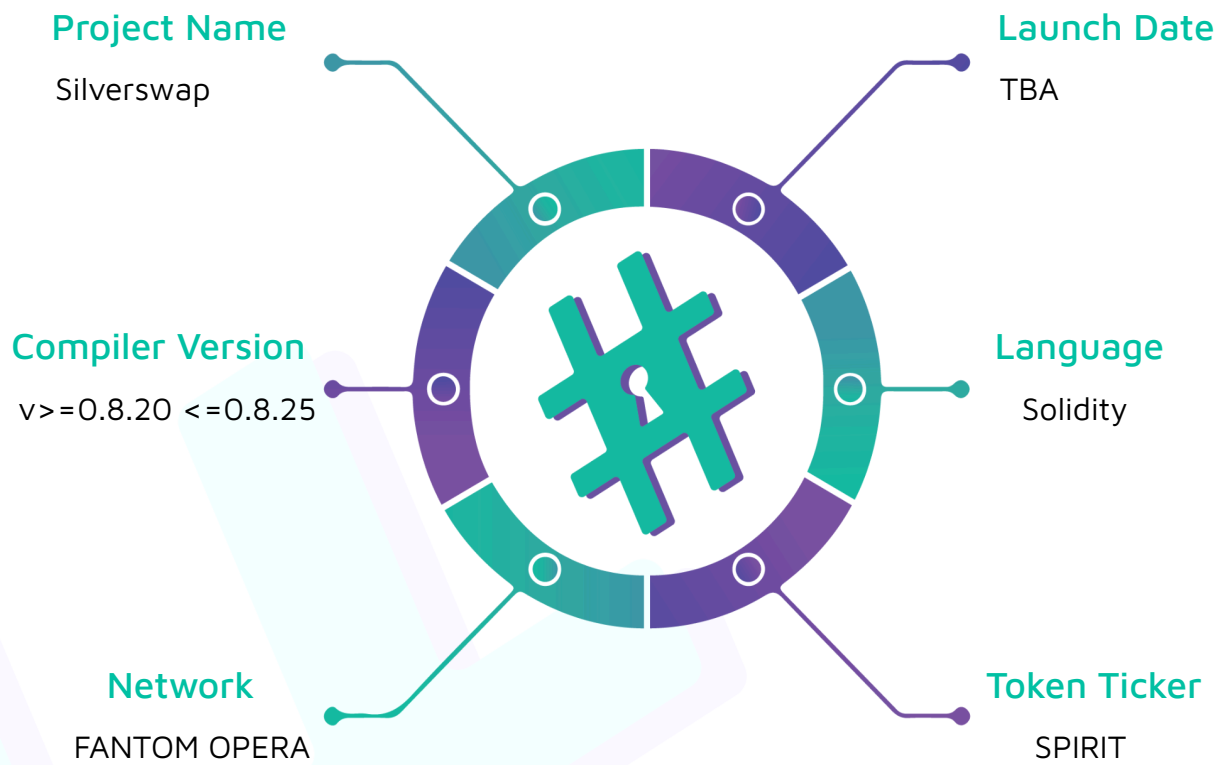
Website: SilverSwap.io

Logo:

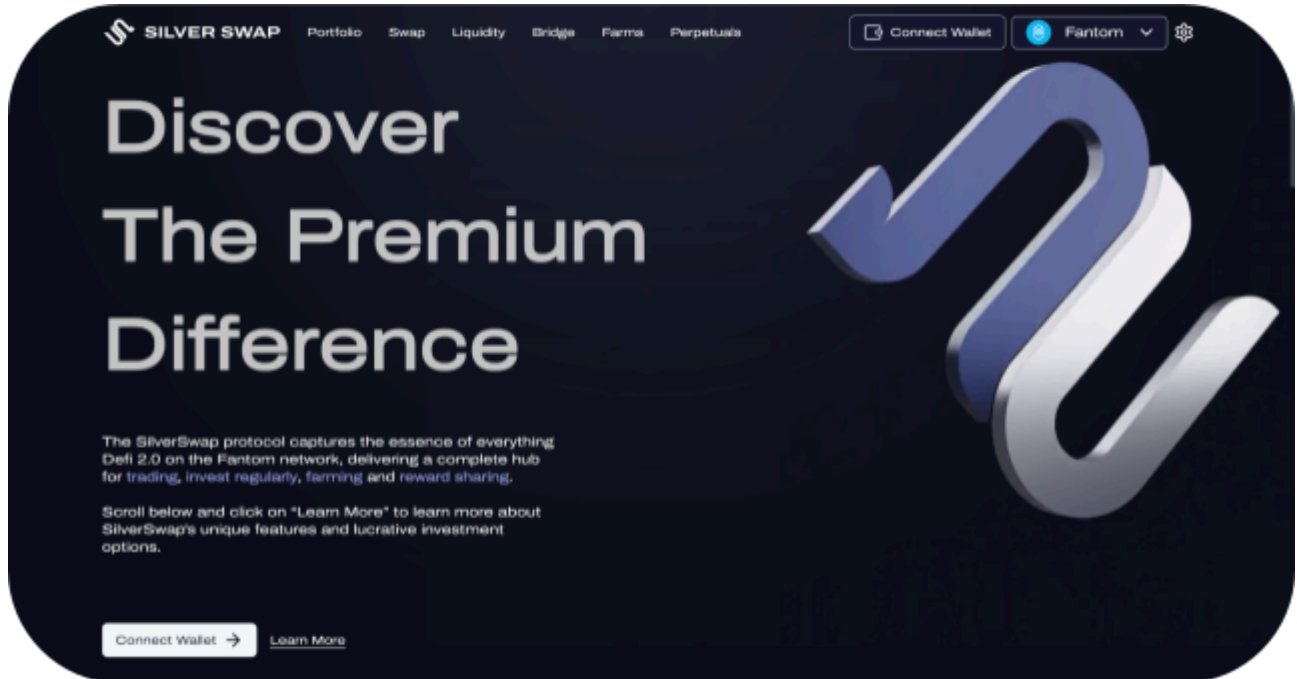


#Hashlock.

Hashlock Pty Ltd

Visualised Context:

Project Visuals:



Audit scope

We at Hashlock audited the solidity code within the Silverswap projects recent changes, the scope of works included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line by line analysis and were supported by software assisted testing.

Description	Silverswap Protocol Smart Contract Changes
Platform	Fantom Opera / Solidity
Audit Date	July, 2024
Contract 1	DcaApprover.sol
Contract 1 MD5 Hash	4abbde3f6f095f1ea2d33b9469deb651
Contract 2	SilverDCA.sol
Contract 2 MD5 Hash	c92b64f491db0b9c991894a7d4831632

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all have correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts. We initially identified some significant vulnerabilities that need to be addressed.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section.

We initially identified some significant vulnerabilities that have since been addressed.

Hashlock found:

0 High severity vulnerabilities

2 Medium severity vulnerabilities

0 Low severity vulnerabilities

2 Gas Optimisations

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Behaviours

Claimed Behaviour	Actual Behaviour
DcaApprover.sol - Contract that gets deployed for each new order users create and handles swaps on the DEX	Contract achieves this functionality.
SilverDCA.sol - Main contract that users interact with: - Deploys a new DcaApprover.sol for each order - Allows the owner to withdraw fees. - Allows users to: - Create new orders - Edit/stop/restart their orders	Contract achieves this functionality.

Code Quality

This Audit scope involves the smart contracts of the Silverswap projects recent changes, as outlined in the Audit Scope section. All contracts, libraries and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however some refactoring was required.

The code is well commented and follows best practice nat-spec styling.

Audit Resources

We were given the Silverswap DCA projects smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The README file and comments are helpful in understanding the overall architecture of the protocol.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

Significance	Description
High	High severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues and inefficiencies

Audit Findings

Medium

[M-01] SilverDCA.sol::executeOrder() - Incorrect logic only allows ftmSwap orders to be executed

Description

In the `executeOrder()` function, execution of swaps will happen differently if the inputted order is an ftmSwap or not. However, due to incorrect logic, any order that is not an ftmSwap will revert the function.

Vulnerability Details

In the `executeOrder()` function shown below:

```
function executeOrder(uint256 id, uint256 amountTokenInGelatoFees,
ExactInputParams memory dcaArgs, ExactInputParams memory ftmSwapArgs) public
onlyOwnerOrDedicatedMsgSender { // H-03 (onlyOwnerOrDedicatedMsgSender)

    //rest of the function

    if (ftmSwapArgs.path.length == 0 || ftmSwapArgs.amountIn == 0 ||
ftmSwapArgs.recipient == address(0)) // G-02

        revert ErrorInvalidExactInputParams();

    bool isFtmSwap = ftmSwapArgs.path.length != 0 && ftmSwapArgs.amountIn != 0
|| ftmSwapArgs.recipient != address(0);

    if (isFtmSwap)
    {

        uint256 gelatoFees = 0;

        require(amountTokenInGelatoFees < ordersById[id].amountIn,
'amountTokenInGelatoFees too high');

        ordersById[id].amountIn -= amountTokenInGelatoFees;

        gelatoFees = ftmSwapArgs.amountIn;
```

```

        ftmSwapArgs.recipient = payable(address(this));

SilverDcaApprover(ordersById[id].approver).transferGelatoFees(gelatoFees);

        TransferHelper.safeApprove(ordersById[id].tokenIn,
address(swapRouter), gelatoFees); // L-06
        //ERC20(ordersById[id].tokenIn).approve(address(proxy),
gelatoFees);

        swapRouter.exactInput(ftmSwapArgs);
    }

    _executeOrder(id, dcaArgs);

    if (isFtmSwap)
    {
        ordersById[id].amountIn = initialAmountIn;
        (uint256 fee, address feeToken) = _getFeeDetails();

        _transfer(fee, feeToken);
        emit GelatoFeesCheck(fee, feeToken);
    }
}

```

It is observed that, if the inputted order is an ftmSwap, this function will execute differently and require a payment of fees. However, due to the highlighted part in the code snippet, any swap that is not ftmSwap will cause the function to revert.

Impact

This vulnerability causes any calls to this function when it is not ftmSwap to fail.

Recommendation

Remove the highlighted revert logic.

Status

Resolved

[M-02] SilverDCA.sol::executeOrder() - Incorrect logic causes orders to incorrectly be counted as an ftmSwap

Description

The `executeOrder()` function will check if an order is an ftmSwap. If so it will execute differently in order to pay fees. However, the check for ftmSwap is incorrectly implemented.

Vulnerability Details

Looking at the `executeOrder()` function shown below:

```
function executeOrder(uint256 id, uint256 amountTokenInGelatoFees,
ExactInputParams memory dcaArgs, ExactInputParams memory ftmSwapArgs) public
onlyOwnerOrDedicatedMsgSender { // H-03 (onlyOwnerOrDedicatedMsgSender)

    //rest of the function

    bool isFtmSwap = ftmSwapArgs.path.length != 0 && ftmSwapArgs.amountIn != 0
    || ftmSwapArgs.recipient != address(0);

    if (isFtmSwap)
    {

        uint256 gelatoFees = 0;

        require(amountTokenInGelatoFees < ordersById[id].amountIn,
'amountTokenInGelatoFees too high');

        ordersById[id].amountIn -= amountTokenInGelatoFees;

        gelatoFees = ftmSwapArgs.amountIn;

        ftmSwapArgs.recipient = payable(address(this));

SilverDcaApprover(ordersById[id].approver).transferGelatoFees(gelatoFees);

        TransferHelper.safeApprove(ordersById[id].tokenIn,
address(swapRouter), gelatoFees); // L-06

        //ERC20(ordersById[id].tokenIn).approve(address(proxy),
gelatoFees);

        swapRouter.exactInput(ftmSwapArgs);
```

```

    }

    _executeOrder(id, dcaArgs);

    if (isFtmSwap)
    {
        ordersById[id].amountIn = initialAmountIn;
        (uint256 fee, address feeToken) = _getFeeDetails();

        _transfer(fee, feeToken);
        emit GelatoFeesCheck(fee, feeToken);
    }
}

```

The highlighted part checks if the inputted swap is an ftmSwap. This logic is implemented incorrectly.

In an example scenario: if the `ftmSwapArgs.path.length` is 0 and the `ftmSwapArgs.amountIn` is 0, if the `ftmSwapArgs.recieipient` is not `address(0)`, the `isFtmSwap` bool will be true. Meaning that even with incorrect inputs, function will execute as if the bool is true.

Impact

Function will calculate the `isFtmSwap` bool incorrectly.

Recommendation

Change the logic as shown below:

```

function executeOrder(uint256 id, uint256 amountTokenInGelatoFees,
ExactInputParams memory dcaArgs, ExactInputParams memory ftmSwapArgs) public
onlyOwnerOrDedicatedMsgSender { // H-03 (onlyOwnerOrDedicatedMsgSender)

    //rest of the function

    bool isFtmSwap = ftmSwapArgs.path.length != 0 && ftmSwapArgs.amountIn != 0
    && ftmSwapArgs.recipient != address(0);

    //rest of the function

```

Status

Resolved

Gas

[G-01] SilverDCA.sol::_executeOrder() - Unnecessary check for minimum amount of tokens received

Description

The `_executeOrder()` function uses the `dcaArgs.amountOutMinimum` input to determine if the user has gotten enough tokens back from the swap. However, checking for this value is unnecessary since the `exactInput()` function called in this function will revert in the following line if the minimum amount out requirement isn't met:

```
function exactInput(ExactInputParams memory params)
    external
    payable
    override
    checkDeadline(params.deadline)
    returns (uint256 amountOut)
{
    //rest of the function

    require(amountOut >= params.amountOutMinimum, 'Too little received');
}
```

Recommendation

Remove the amountOutMin calculation in the `_executeOrder()` function.

Status

Resolved

[G-02] SilverDCA.sol::executeOrder() - Function will continue on with execution if deadline has passed resulting in a waste of gas

Description

Since there are no checks for the deadline in the function itself, this check will only be done when the function calls the `exactInput()` function. If the deadline requirement isn't met, the function will execute up to this line and will only revert here. This results in calls to this function wasting gas.

Recommendation

Implement a deadline check in the `executeOrder()` function, an example given below:

```
function executeOrder(uint256 id, uint256 amountTokenInGelatoFees,
ExactInputParams memory dcaArgs, ExactInputParams memory ftmSwapArgs) public
onlyOwnerOrDedicatedMsgSender { // H-03 (onlyOwnerOrDedicatedMsgSender)

    if (dcaArgs.deadline < block.timestamp){

        revert;

    }

    //rest of the function
```

Status

Resolved

Centralisation

The Silverswap project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlocks analysis, the Silverswap project seems to have a sound and well tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits are to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contracts details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au

#Hashlock.



#Hashlock.

Hashlock Pty Ltd